

Music Data Preprocessing Pipelines: A Hands-On Tutorial

Alexander SIGMAN

Munich Music Labs Knowledge Sharing Session

19 February 2026

MOTIVATION

- **Model architectures:** receive maximal attention (pun intended)
- **Data Pipelines:** often overlooked
- **Music data:** unique challenges (especially, but not exclusively, in the audio domain)
- A model can only generate data that is as high-quality as the data to which it has been exposed
- Data quality and representation can heavily influence the success/failure of a given model



PERSONAL EXPERIENCE

- Data analysis/understanding ➡ model evaluation/diagnostics/subjective test interpretation
- “Source of truth”: not always only one, not always consistent
- Data pipeline design: complex and iterative process...but aim for simplicity and elegance
- Short-term dev cost ➡ long-term dividends
- Always Be Testing: numerous potential sources of error



WHAT THIS TUTORIAL WILL ****NOT**** COVER

- **Database architectures (SQL, Mongo, Redis, vector DBs, etc.)**
- **Handling real-time/streaming data (Kafka)**
- **Deployment and monitoring**
- [These are important, but out of scope: more data engineering/MLOps-related]



WHAT THIS TUTORIAL * * WILL * * COVER

- **Audio preprocessing**
- **Text tokenisation/preprocessing**
- **Data validation**
- **Pipeline design decisions** (and their complexities/trade-offs)
- **Multimodal dataset construction and training loop data loading**
- **Audio representations** (waveforms, spectrograms, embeddings...)
- **Appropriate tests and sanity checks**
- **Practical resource/compute considerations and trade-offs**



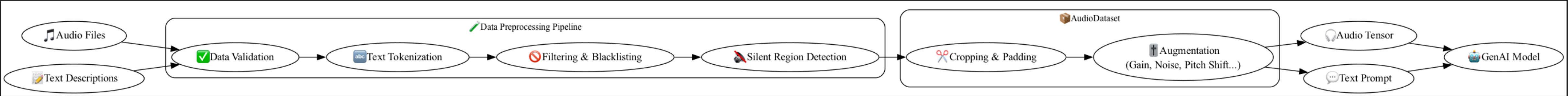
USEFUL EXPERIENCE

- **Basic Pandas familiarity**
- **PyTorch/Torchaudio functionality**
 - **nothing too fancy: the focus is on data manipulation, not models**
- **Basic audio DSP concepts**



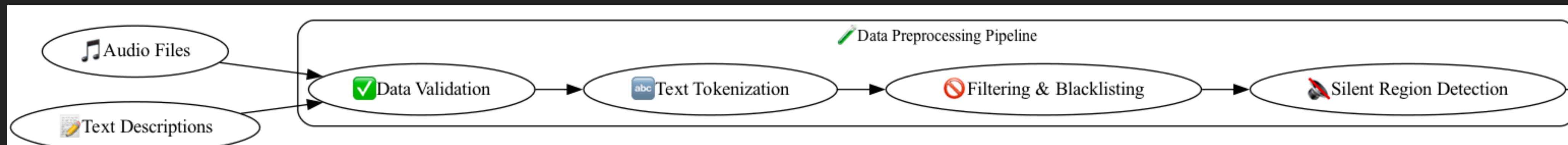
PROJECT STRUCTURE/OBJECTIVES

- **Task:** Prepare training data (audio with text annotations) for training a generative music model



PROJECT STRUCTURE/OBJECTIVES

- **Task:** Prepare training data (audio with text annotations) for training a generative music model
- **Part I: Data Preprocessing Pipeline**
 - Input: Toy dataset of text annotations and audio paths (CSV)
 - Output: Data entries dictionary (serialized to JSON, for demonstration purposes)



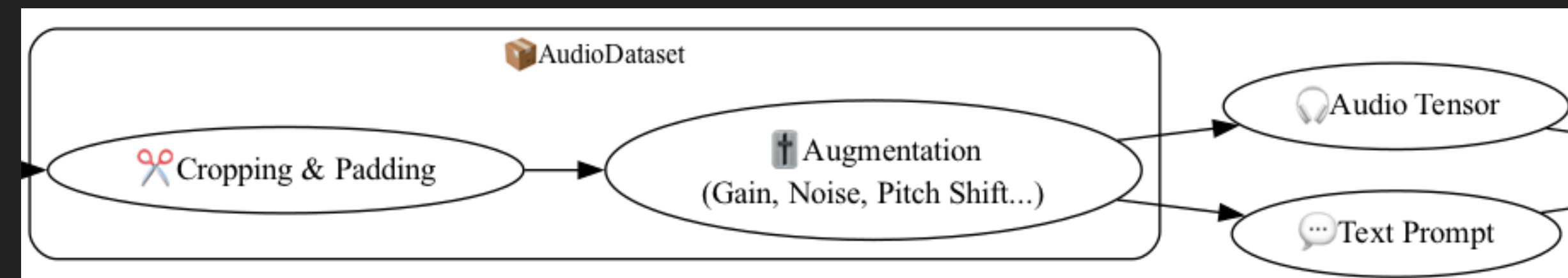
PROJECT STRUCTURE/OBJECTIVES

- **Task:** Prepare training data (audio with text annotations) for training a generative music model

- **Part I: Data Preprocessing Pipeline**

- **Part II: Audio Dataset and DataLoader**

- Input: Above data entries (filtered)
- Dataset operations:
 - Random cropping + (non-random) padding
 - Apply augmentations
 - Optionally apply audio transform
 - Compile TextCondition
- Output: Model-ready training and validation DataLoaders



RESOURCES

- **Colab notebook:** https://colab.research.google.com/drive/1Kgb7JkHDWP9FfQylaYjP2Lxgu9_2Sasr?usp=sharing
- **Github repo:** https://github.com/atsigman/data_pipeline_tutorial



INSTRUCTIONS

- 1) **Open the notebook in Google Colab:** just click on  [Open in Colab](#)
- 2) Follow the directions in the notebook
- 3) Keep the **Github repo** open for reference (there will be links to specific functions and modules in the notebook)



FINAL TIPS

- Keep designs *modular* and *flexible*: do not require reinventing the wheel if anything changes in the future

FINAL TIPS

- Keep designs *modular* and *flexible*: do not require reinventing the wheel if anything changes in the future
- For an offline data pipeline run locally, it *may* be worth remaining in numpy/librosa space, and avoiding the “burden” of PyTorch. However, this comes at an efficiency cost (particularly regarding processing times...)

FINAL TIPS

- Keep designs *modular* and *flexible*: do not require reinventing the wheel if anything changes in the future
- For an offline data pipeline run locally, it *may* be worth remaining in numpy/librosa space, and avoiding the “burden” of PyTorch. However, this comes at an efficiency cost (particularly regarding processing times...)
- For any online (audio) array operations (i.e., executed during training): opt for torch/torchaudio over numpy/librosa

FINAL TIPS

- Keep designs *modular* and *flexible*: do not require reinventing the wheel if anything changes in the future
- For an offline data pipeline run locally, it *may* be worth remaining in numpy/librosa space, and avoiding the “burden” of PyTorch. However, this comes at an efficiency cost (particularly regarding processing times...)
- For any online operations (i.e., executed during training): opt for torch/torchaudio over numpy/librosa
- Introduce *meaningful* optimisations/heuristics where possible. This can accelerate processes by an order of magnitude. Such optimisations include:
 - Using a specific backend for loading audio based upon file extension, if possible
 - If the source sample rate is consistent, using `torchaudio.transforms.Resample` (instantiated once), as opposed to `torchaudio.functional.resample`
 - Downmixing and cropping audio ASAP
 - Caching artefacts as little as possible
 - Precomputing audio embeddings or other compressed representations

CODA: E2E PIPELINE SCRIPT

- https://github.com/atsigman/data_pipeline_tutorial/blob/main/src/scripts/run_pipeline.py
 - [accepts CSV or JSON input]

THANKS (AGAIN)!

- **Questions?**